# R3.07
# SQL dans un langage de programmation

## Chapitre 2

## Transactions et concurrence d'accès




Selena Lamari et Chouki Tibermacine

BUT 2 Informatique — IUT de Vannes — Université Bretagne Sud

# Plan du cours

1. Transactions : rappels
2. Concurrence d’accès
3. Niveaux d’isolation
4. Éviter les mises à jour perdues
5. Interblocages
6. Contraintes, erreurs et bonnes pratiques
7. Exigences pour les rendus de TP

# Qu’est-ce qu’une transaction ?

**Définition**

Une **transaction** est une suite d’opérations sur la base qui forme une **unité logique de travail** : soit tout est appliqué (`COMMIT`), soit rien (`ROLLBACK`).

**Propriétés ACID**

- **A**tomicité : tout ou rien
- **C**ohérence : les contraintes restent vraies
- **I**solation : les transactions concurrentes ne se gênent pas (à quel point ?)
- **D**urabilité : un `COMMIT` survit à une panne

**Exemple : commande en ligne**

1. retirer les articles du stock
2. créer la commande
3. journaliser l’opération

Une panne entre 1 et 2 ne doit pas faire « disparaître » des articles du stock.

# Transactions en JDBC : le schéma à retenir

```
try (Connection c = dataSource.getConnection()) {
  c.setAutoCommit(false);              // par defaut : autocommit apres CHAQUE requete
  try {
    retirerStock(c, produit, quantite);
    creerCommande(c, client, produit, quantite);
    c.commit();                        // tout est valide
  } catch (SQLException e) {
    c.rollback();                      // rien n'est applique
    throw e;
  }
}                                      // close() rend la connexion au pool
```

- Toutes les requêtes d'une transaction utilisent la même `Connection`
- Points de reprise partiels possibles : `Savepoint sp = c.setSavepoint(); ... c.rollback(sp);`

# D'où vient la `Connection` ?

**En TP : `DriverManager`**

```
Connection c = DriverManager
  .getConnection(url, user, pwd);
// nouvelle connexion physique
```

**En production : un pool**

```
Connection c = dataSource
  .getConnection();   // empruntee
c.close();            // rendue au pool
```

- Ouvrir une connexion coûte cher : un **pool** (ex. HikariCP) en garde quelques-unes prêtes
- Le code transactionnel (`setAutoCommit`, `commit`, `rollback`) est **identique**
- Les slides de ce chapitre écrivent `dataSource.getConnection()` : en TP, utilisez votre `getConnection()`

# Plan du cours

1. Transactions : rappels
2. Concurrence d’accès
3. Niveaux d’isolation
4. Éviter les mises à jour perdues
5. Interblocages
6. Contraintes, erreurs et bonnes pratiques
7. Exigences pour les rendus de TP

# Pourquoi la concurrence est inévitable

- Une application web ou mobile sert **plusieurs utilisateurs en même temps**
- Chaque requête HTTP est traitée par un **thread** différent, avec sa propre connexion (pool)
- Plusieurs transactions lisent et écrivent donc les **mêmes lignes** au même moment

**Question**

Deux clients commandent le même article (stock : 10) au même instant : $-2$ et $-3$. Quel est le stock final ?

# Pourquoi la concurrence est inévitable

- Une application web ou mobile sert **plusieurs utilisateurs en même temps**
- Chaque requête HTTP est traitée par un **thread** différent, avec sa propre connexion (pool)
- Plusieurs transactions lisent et écrivent donc les **mêmes lignes** au même moment

**Question**

Deux clients commandent le même article (stock : 10) au même instant : $-2$ et $-3$. Quel est le stock final ? Pas forcément 5…

# L’anomalie la plus courante : la mise à jour perdue



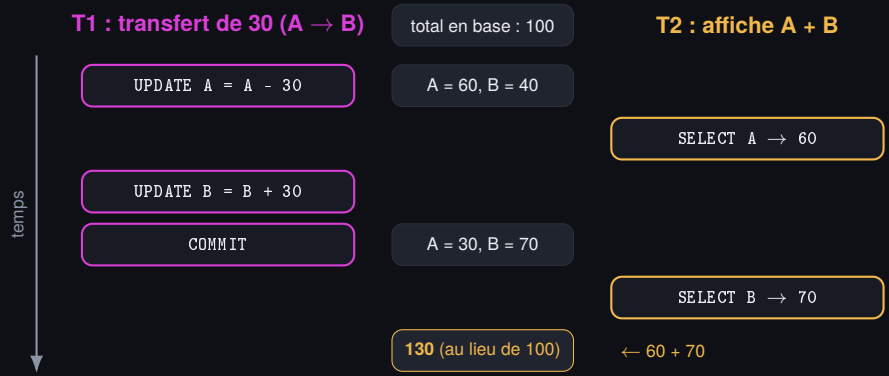

- Chaque transaction calcule le nouveau stock **dans le code Java** à partir d’une valeur lue trop tôt
- C’est le niveau d’isolation **par défaut** de PostgreSQL (`READ COMMITTED`) : testé, résultat 8

# Les anomalies de concurrence

| Anomalie | Ce qui se passe |
|---|---|
| Lecture sale<br>*dirty read* (P1) | T1 lit une valeur modifiée par T2 **non encore validée** ; T2 fait ensuite `ROLLBACK` |
| Lecture non répétable<br>*non-repeatable read* (P2) | T1 lit deux fois la même ligne et obtient deux valeurs différentes (T2 a validé entre-temps) |
| Lecture fantôme<br>*phantom read* (P3) | T1 relance une requête et voit des **lignes en plus ou en moins** (insertion/suppression de T2) |
| Mise à jour perdue<br>*lost update* (P4) | T1 et T2 lisent puis écrivent la même ligne : l'écriture de l'une écrase celle de l'autre |

P1 à P3 : phénomènes de la norme ANSI SQL-92. P4 : ajouté par Berenson et al. (SIGMOD 1995), absent de la norme.

# Exemple : une lecture incohérente (*read skew*)




- A et B : deux entrepôts. PostgreSQL ne lit **jamais** une valeur non validée, mais deux `SELECT` séparés peuvent voir **deux états** de la base
- Remède : **une seule requête** (`SELECT SUM(stock) ...`) ou `REPEATABLE READ`

# Plan du cours

1. Transactions : rappels
2. Concurrence d’accès
3. Niveaux d’isolation
4. Éviter les mises à jour perdues
5. Interblocages
6. Contraintes, erreurs et bonnes pratiques
7. Exigences pour les rendus de TP

# Les quatre niveaux d'isolation

| Niveau | Lect. sale | Non répétable | Fantôme | Màj perdue |
|---|---|---|---|---|
| `READ UNCOMMITTED` | possible* | possible | possible | possible |
| `READ COMMITTED` (défaut PG) | — | possible | possible | possible |
| `REPEATABLE READ` | — | — | possible* | détectée |
| `SERIALIZABLE` | — | — | — | détectée |

* selon la norme. PostgreSQL est plus strict : `READ UNCOMMITTED` s'y comporte comme `READ COMMITTED` (jamais de lecture sale), et `REPEATABLE READ` y empêche aussi les fantômes. La colonne « Màj perdue » n'est pas dans la norme (P4, Berenson et al. 1995). Source : documentation PostgreSQL, §13.2.

- Plus le niveau est élevé, plus il y a de transactions **refusées** ou **bloquées** : on paie l'isolation en débit
- PostgreSQL utilise le **MVCC** : chaque transaction lit une « photo » (snapshot) de la base ; les lecteurs ne bloquent jamais les écrivains

# Choisir le niveau d'isolation

### En SQL

```
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT stock FROM Produit
 WHERE numProduit = 1;
UPDATE Produit SET stock = 8
 WHERE numProduit = 1;
COMMIT;
```

### En JDBC

```
c.setAutoCommit(false);
c.setTransactionIsolation(
  Connection.TRANSACTION_REPEATABLE_READ);
// ... requetes ...
c.commit();
```

**Ce qu'on a observé sur PostgreSQL (scénario des deux clients)**

- `READ COMMITTED` : stock final 8 (au lieu de 5) — mise à jour perdue
- `REPEATABLE READ` : T1 reçoit `ERROR: could not serialize access due to concurrent update` — SQLState `40001`

⇒ à un niveau élevé, l'application doit **rejouer** la transaction refusée

# Plan du cours

1. Transactions : rappels
2. Concurrence d’accès
3. Niveaux d’isolation
4. Éviter les mises à jour perdues
5. Interblocages
6. Contraintes, erreurs et bonnes pratiques
7. Exigences pour les rendus de TP

# Solution 1 : laisser la base faire le calcul

```java
// NON : lecture, calcul en Java, puis ecriture (2 requetes, fenetre de concurrence)
int stock = lireStock(c, num);
majStock(c, num, stock - quantite);

// OUI : une seule requete, atomique pour la ligne concernee
PreparedStatement ps = c.prepareStatement(
  "UPDATE Produit SET stock = stock - ? WHERE numProduit = ? AND stock >= ?");
ps.setInt(1, quantite); ps.setInt(2, num); ps.setInt(3, quantite);
if (ps.executeUpdate() == 0) throw new StockInsuffisantException(num);
```

- La 2e transaction attend le verrou de ligne, puis relit la valeur à jour : résultat 5
- La condition `stock >= ?` vérifie la règle métier **dans la même requête**
- À privilégier dès que le calcul s'exprime en SQL

# Les verrous : partagé (S) et exclusif (X)

**Compatibilité**

| | déjà S | déjà X |
|---|---|---|
| **veut S** | ok | attend |
| **veut X** | attend | attend |

**Dans PostgreSQL (par ligne)**

`SELECT` simple : aucun verrou

`FOR SHARE` : verrou S

`UPDATE`, `DELETE`, `FOR UPDATE` : verrou X

- Les lecteurs simples ne bloquent jamais les écrivains (MVCC)
- Verrouillage à deux phases **strict** : les verrous sont gardés jusqu'au `COMMIT` ou `ROLLBACK`
- D'où : transactions **courtes**, et toujours le **même ordre** de verrouillage

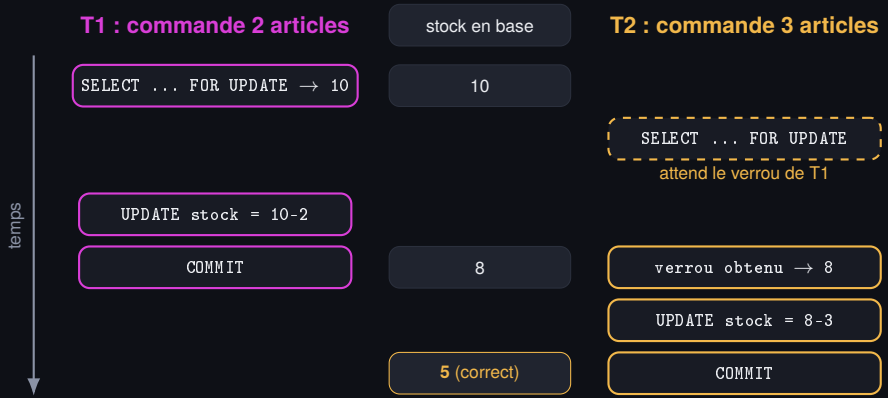
Source : documentation PostgreSQL, §13.3 « Explicit Locking » (verrous de ligne, interblocages).

# Solution 2 : verrou pessimiste (SELECT ... FOR UPDATE)

```
c.setAutoCommit(false);
PreparedStatement sel = c.prepareStatement(
  "SELECT stock FROM Produit WHERE numProduit = ? FOR UPDATE");  // pose un verrou
sel.setInt(1, num);
ResultSet rs = sel.executeQuery(); rs.next();
int stock = rs.getInt("stock");
// ... regles metier complexes en Java ...
majStock(c, num, stock - quantite);
c.commit();                                                    // libere le verrou
```

- Les autres transactions qui veulent la même ligne **attendent** le COMMIT
- Utile quand la logique est trop complexe pour une seule requête
- Transaction **courte** obligatoire : jamais d'attente utilisateur pendant qu'on tient un verrou

# Avec `FOR UPDATE`, la 2e transaction attend




- `READ COMMITTED` (défaut) : T2 est **bloquée** jusqu'au `COMMIT` de T1, puis lit la valeur **à jour**
- Prix à payer : les deux transactions s'exécutent l'une après l'autre
- `REPEATABLE READ` : T2 recevrait plutôt l'erreur `40001`, à rejouer

# Solution 3 : verrou optimiste (colonne de version)

```sql
ALTER TABLE Produit ADD COLUMN version INT NOT NULL DEFAULT 0;
-- lecture : on memorise la version lue (ex. 7)
SELECT stock, version FROM Produit WHERE numProduit = 1;
-- ecriture : seulement si personne n'a modifie la ligne entre-temps
UPDATE Produit SET stock = 8, version = version + 1
 WHERE numProduit = 1 AND version = 7;
```

- 0 ligne modifiée $\Rightarrow$ conflit : on prévient l'utilisateur ou on rejoue
- Aucun verrou tenu entre la lecture et l'écriture : adapté aux **longues interactions** (formulaire web)
- JPA l'automatise avec `@Version` (chapitre 6)

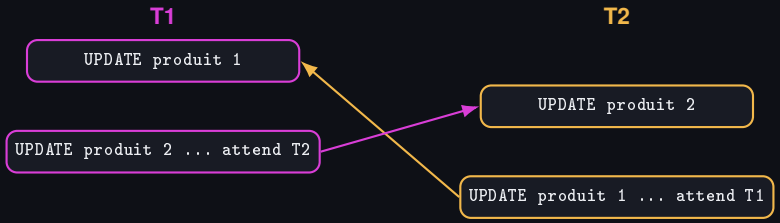
# Quelle solution choisir ?

| Approche | Quand l'utiliser | Prix à payer |
|---|---|---|
| Calcul en SQL | mise à jour simple (compteur, stock) | aucun |
| `FOR UPDATE` | logique complexe, conflits fréquents | attente, risque d'interblocage |
| Version (optimiste) | conflits rares, interaction longue | gérer l'échec (message, rejeu) |
| `REPEATABLE READ`/ `SERIALIZABLE` | règles portant sur plusieurs lignes | rejouer sur SQLState `40001` |

# Plan du cours

1. Transactions : rappels
2. Concurrence d’accès
3. Niveaux d’isolation
4. Éviter les mises à jour perdues
5. Interblocages
6. Contraintes, erreurs et bonnes pratiques
7. Exigences pour les rendus de TP

# Interblocage (*deadlock*)




- PostgreSQL détecte le cycle (au bout d'une seconde) et **annule l'une des deux** : `ERROR: deadlock detected` SQLState `40P01`
- Prévention : toujours verrouiller dans le **même ordre** (ex. produit de plus petit numéro d'abord), transactions courtes
- L'application traite `40P01` comme `40001` : on **rejoue** la transaction

# Rejouer une transaction refusée

```
for (int essai = 1; ; essai++) {
  try (Connection c = dataSource.getConnection()) {
    c.setAutoCommit(false);
    c.setTransactionIsolation(Connection.TRANSACTION_REPEATABLE_READ);
    try {
      passerCommande(c, client, panier);
      c.commit();
      return;
    } catch (SQLException e) {
      c.rollback();
      String etat = e.getSQLState();
      boolean rejouable = "40001".equals(etat) || "40P01".equals(etat);
      if (!rejouable || essai == 3) throw e;
    }
  }
}
```

# Plan du cours

1. Transactions : rappels
2. Concurrence d’accès
3. Niveaux d’isolation
4. Éviter les mises à jour perdues
5. Interblocages
6. Contraintes, erreurs et bonnes pratiques
7. Exigences pour les rendus de TP

# Contraintes déclaratives

```
ALTER TABLE Etudiant ALTER COLUMN nom SET NOT NULL;
ALTER TABLE Etudiant ADD CONSTRAINT chk_moy CHECK (moyenne BETWEEN 0 AND 20);
ALTER TABLE Etudiant ADD CONSTRAINT uq_mail UNIQUE (email);
ALTER TABLE Inscription ALTER COLUMN statut SET DEFAULT 'en attente';
```

- `NOT NULL`, `CHECK`, `UNIQUE`, clés primaires et étrangères : la **base refuse** toute ligne qui les viole
- `DEFAULT` n'interdit rien : il fournit la valeur utilisée quand la colonne est omise à l'`INSERT`
- `ALTER TABLE` modifie le schéma sans le recréer ; ajouter un `NOT NULL`, un `CHECK` ou un `UNIQUE` échoue si des lignes existantes violent déjà la règle
- Relation plusieurs-à-plusieurs : **table d'association** (ex. `Inscription(numEtudiant, numCours)`) à clé primaire composite

# Erreurs SQL : lire le `SQLState`

| SQLState | Signification | Réaction |
|---|---|---|
| `23505` | doublon (clé primaire, `UNIQUE`) | erreur métier |
| `23503` | clé étrangère inexistante | erreur métier |
| `23514` | `CHECK` violé (ex. `moyenne > 20`) | erreur métier |
| `40001` | échec de sérialisation | **rejouer** |
| `40P01` | interblocage détecté | **rejouer** |

```
catch (SQLException e) {
  c.rollback();
  if ("23505".equals(e.getSQLState()))
    throw new EtudiantExistantException(num);
  throw e;
}
```

- `PSQLException` n'a pas de sous-classe dédiée : on lit `getSQLState()`

Source : documentation PostgreSQL, annexe A « PostgreSQL Error Codes » et §13.5.

# Bonnes pratiques

- Des transactions **courtes** : pas d'appel réseau, pas d'attente utilisateur à l'intérieur
- La frontière de transaction est dans la **couche service** (chapitre 5), pas dans chaque DAO
- Préférer les mises à jour **relatives** en SQL (`stock = stock - ?`)
- Faire vérifier les règles par la base : `CHECK (stock >= 0)`, `UNIQUE`, clés étrangères
- Montants : `NUMERIC` en SQL, `BigDecimal` en Java : `REAL`/`double` sont inexacts ($0{,}1 + 0{,}2 \neq 0{,}3$)
- Toujours `rollback()` en cas d'exception, et `close()` pour rendre la connexion au pool

# Plan du cours

1. Transactions : rappels
2. Concurrence d’accès
3. Niveaux d’isolation
4. Éviter les mises à jour perdues
5. Interblocages
6. Contraintes, erreurs et bonnes pratiques
7. Exigences pour les rendus de TP

# Secrets : le fichier `.env`

**.env** (jamais dans Git)

```
POSTGRES_USER=app
POSTGRES_PASSWORD=change-me
POSTGRES_DB=campus_db
DB_URL=jdbc:postgresql://db:5432/campus_db
DB_USER=app
DB_PASSWORD=change-me
```

**Lecture côté Java**

```
// conteneur : injecte par Docker
System.getenv("DB_URL");
// hors Docker : dotenv-java
Dotenv.configure()
  .ignoreIfMissing().load()
  .get("DB_URL");
```

- `.env` dans `.gitignore` ; on versionne un `.env.example` sans secret
- Docker Compose injecte `.env` dans le conteneur (`env_file`)
- Hors Docker (IntelliJ) : l'hôte `db` devient `localhost`

# Un rendu de qualité, c'est…

**Dépôt Git**

- Un dépôt par binôme, commits réguliers et messages explicites
- `.gitignore` : `target/`, `.idea/`, `.env`
- Aucun mot de passe dans le code ni dans l'historique
- Projet Maven standard (`src/main/java`, `src/test/java`)

**README.md**

- Comment lancer la base (`cp .env.example .env`, puis `docker compose up -d`)
- Comment compiler et exécuter (`mvn ...`)
- Ce qui est fait, ce qui ne l'est pas
- Choix techniques et difficultés

**Test d'acceptation**

Un enseignant clone le dépôt sur une machine vierge et suit le README : **ça doit fonctionner**.

# Références (1/2) : articles et livre


- H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil, P. O'Neil. *A Critique of ANSI SQL Isolation Levels*. ACM SIGMOD 1995, pp. 1–10. https://doi.org/10.1145/223784.223785
  Phénomènes P1–P4, *read skew*, *snapshot isolation*.
- T. Härder, A. Reuter. *Principles of Transaction-Oriented Database Recovery*. ACM Computing Surveys 15(4), 1983, pp. 287–317. https://doi.org/10.1145/289.291
  Référence classique sur les transactions et la reprise après panne.
- M. Kleppmann. *Designing Data-Intensive Applications*. O'Reilly, 2017. https://dataintensive.net/
  Chapitre 7 « Transactions » : isolation, mises à jour perdues, *write skew*.

# Références (2/2) : documentation et outils

- PostgreSQL, chapitre 13 « Concurrency Control » : §13.2 *Transaction Isolation*, §13.3 *Explicit Locking*, §13.5 *Serialization Failure Handling*
  `https://www.postgresql.org/docs/current/transaction-iso.html`
  `https://www.postgresql.org/docs/current/explicit-locking.html`
  `https://www.postgresql.org/docs/current/mvcc-serialization-failure-handling.html`
- PostgreSQL, annexe A « PostgreSQL Error Codes » (SQLState)
  `https://www.postgresql.org/docs/current/errcodes-appendix.html`
- Oracle, *The Java Tutorials* : « Using Transactions » (JDBC)
  `https://docs.oracle.com/javase/tutorial/jdbc/basics/transactions.html`
- Pilote JDBC PostgreSQL : classe `PSQLException`
  `https://jdbc.postgresql.org/documentation/publicapi/org/postgresql/util/PSQLException.html`
- Outils : HikariCP `https://github.com/brettwooldridge/HikariCP` ; dotenv-java `https://github.com/cdimascio/dotenv-java` ; Docker Compose, `env_file` `https://docs.docker.com/compose/how-tos/environment-variables/set-environment-variables/`

# Questions ?



Chouki.Tibermacine@univ-ubs.fr